

1. Find Maximum of Three Numbers

```
def find_max(a, b, c):  
    return max(a, b, c)  
  
num1 = int(input("Enter first number: "))  
num2 = int(input("Enter second number: "))  
num3 = int(input("Enter third number: "))  
print("The maximum number is:", find_max(num1, num2, num3))
```

Explanation:

- **Logic:** The function `find_max` uses the built-in `max()` function to determine the largest of three numbers.
- **Keywords:** `def`, `return`, `input`, `print`, `max()`
- **Debugging Tip:** Ensure the inputs are integers. If you get an error, check the input type.

Sample Input:

```
Enter first number: 15  
Enter second number: 22  
Enter third number: 7
```

Sample Output:

```
The maximum number is: 22
```

2. Basic Structure of a Python Program

```
def greet(name):  
    print(f"Hello, {name}!")
```

```
user_name = input("Enter your name: ")
greet(user_name)
```

Explanation:

- **Logic:** The program defines a function `greet` that takes a name as input and prints a greeting.
- **Keywords:** `def`, `print`, `input`
- **Debugging Tip:** Check if the input is empty before calling the function.

Sample Input:

mathematica

Enter your name: Alice

Sample Output:

Hello, Alice!

3. Simple Calculator Using Functions

python

```
def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def calculator():
    print("Select operation: 1. Add 2. Subtract")
    choice = input("Enter choice (1/2): ")
    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))

    if choice == '1':
        print("Result:", add(num1, num2))
    elif choice == '2':
        print("Result:", subtract(num1, num2))
    else:
```

```
        print("Invalid choice")

calculator()
```

Explanation:

- **Logic:** This program prompts the user to select an operation and input two numbers. It then performs the chosen calculation using defined functions.
- **Keywords:** `def`, `print`, `input`, `if`, `elif`, `else`, `float()`
- **Debugging Tip:** Check if the operation selected is valid. Handle exceptions for non-numeric inputs.

Sample Input:

mathematica

```
Select operation: 1. Add 2. Subtract
Enter choice (1/2): 1
Enter first number: 5
Enter second number: 10
```

•

Sample Output:

```
Result: 15.0
```

•

4. Data Types Demonstration

python

```
int_var = 10
float_var = 10.5
string_var = "Hello"
list_var = [1, 2, 3]
tuple_var = (1, 2, 3)
dict_var = {"key": "value"}
set_var = {1, 2, 3}
```

```
print(f"Integer: {int_var}, Float: {float_var}, String: {string_var}")
print(f"List: {list_var}, Tuple: {tuple_var}, Dictionary: {dict_var},
Set: {set_var}")
```

Explanation:

- **Logic:** This program initializes variables of different data types and prints their values.
- **Keywords:** `print`, `f-string`
- **Debugging Tip:** If a variable is not displaying correctly, ensure it is initialized properly.

Sample Output:

yaml

```
Integer: 10, Float: 10.5, String: Hello
List: [1, 2, 3], Tuple: (1, 2, 3), Dictionary: {'key': 'value'}, Set:
{1, 2, 3}
```

•

5. Type Conversion Example

python

```
string_num = input("Enter a number: ")
try:
    converted_num = int(string_num)
    print("Converted number:", converted_num)
    print("Type of converted number:", type(converted_num))
except ValueError:
    print("Invalid input! Please enter a numeric value.")
```

Explanation:

- **Logic:** The program converts a string input to an integer and prints its value and type. It handles invalid inputs using a try-except block.
- **Keywords:** `input`, `try`, `except`, `int()`, `print`, `type()`
- **Debugging Tip:** If you receive a `ValueError`, check if the input is a valid number.

Sample Input:

css

```
Enter a number: 42
```

-

Sample Output:

```
python
```

```
Converted number: 42
```

```
Type of converted number: <class 'int'>
```

-

6. Input-Output Functions

```
python
```

```
name = input("Enter your name: ")
age = input("Enter your age: ")
print(f"Name: {name}, Age: {age}")
```

Explanation:

- **Logic:** Captures user input for name and age, formatted in a string.
- **Keywords:** `input`, `print`, `f-string`
- **Debugging Tip:** Ensure inputs are correctly received; check for whitespace or unexpected characters.

Sample Input:

```
mathematica
```

```
Enter your name: John
```

```
Enter your age: 30
```

-

Sample Output:

```
yaml
```

```
Name: John, Age: 30
```

-

7. Operator Precedence Example

python

```
a = 10
b = 20
c = 30
result = a + b * c / 10 - 5 # Operator precedence applied
print("Result of expression:", result)
```

Explanation:

- **Logic:** Demonstrates how Python applies operator precedence in arithmetic expressions.
- **Keywords:** `print`
- **Debugging Tip:** Break down complex expressions to validate intermediate results.

Sample Output:

r

```
Result of expression: 55.0
```

-

8. Grading System Using Conditional Statements

python

```
marks = float(input("Enter your marks: "))
if marks >= 90:
    grade = 'A'
elif marks >= 80:
    grade = 'B'
elif marks >= 70:
    grade = 'C'
elif marks >= 60:
    grade = 'D'
else:
    grade = 'F'
```

```
print("Your grade is:", grade)
```

Explanation:

- **Logic:** This program evaluates marks and assigns a grade based on defined criteria.
- **Keywords:** `if`, `elif`, `else`, `print`, `input`
- **Debugging Tip:** Check for correct comparison logic to ensure grades are assigned correctly.

Sample Input:

CSS

```
Enter your marks: 85
```

-

Sample Output:

```
Your grade is: B
```

-

9. Looping through a List

python

```
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)
```

Explanation:

- **Logic:** Iterates over a list of fruits and prints each one.
- **Keywords:** `for`, `print`
- **Debugging Tip:** Ensure the list is not empty; check the loop variable in case of errors.

Sample Output:

```
apple  
banana
```

cherry

-

10. While Loop to Count Down

python

```
count = 5
while count > 0:
    print(count)
    count -= 1
print("Lift off!")
```

Explanation:

- **Logic:** A countdown from 5 to 1, using a while loop, followed by a message.
- **Keywords:** `while`, `print`
- **Debugging Tip:** Ensure the loop has a terminating condition; watch for infinite loops.

Sample Output:

```
5
4
3
2
1
Lift off!
```

-

11. Nested Loops for Multiplication Table

python

```
num = int(input("Enter a number: "))
for i in range(1, 11):
    for j in range(1, 11):
```

```
print(f"{i} x {j} = {i*j}")
```

Explanation:

- **Logic:** Generates a multiplication table for the input number using nested loops.
- **Keywords:** `for`, `print`, `input`
- **Debugging Tip:** Verify that the ranges are set correctly for desired multiplication range.

Sample Input:

CSS

Enter a number: 3

•

Sample Output:

python

1 x 1 = 1

1 x 2 = 2

...

3 x 10 = 30

•

12. List Comprehensions

python

```
squares = [x**2 for x in range(10)]
```

```
print("Squares:", squares)
```

Explanation:

- **Logic:** Creates a list of squares for numbers from 0 to 9 using list comprehension.
- **Keywords:** `for`, `print`
- **Debugging Tip:** Ensure the range is correct to avoid off-by-one errors.

Sample Output:

Squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

-

13. Dictionary Example

python

```
person = {"name": "Alice", "age": 25}
print(f"Name: {person['name']}, Age: {person['age']}")
```

Explanation:

- **Logic:** Initializes a dictionary and accesses its values using keys.
- **Keywords:** `print`
- **Debugging Tip:** Ensure keys are spelled correctly; watch for `KeyErrors`.

Sample Output:

yaml

```
Name: Alice, Age: 25
```

-

14. Set Operations

python

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
union_set = set1 | set2
print("Union:", union_set)
```

Explanation:

- **Logic:** Demonstrates set union using the `|` operator.
- **Keywords:** `print`
- **Debugging Tip:** Check for duplicates; sets automatically handle them.

Sample Output:

CSS

Union: {1, 2, 3, 4, 5}

-
-

15. File Handling: Writing to a File

python

```
with open("sample.txt", "w") as file:  
    file.write("Hello, World!")
```

Explanation:

- **Logic:** Opens a file in write mode and writes a string to it using a context manager.
 - **Keywords:** `with`, `open`, `write`
 - **Debugging Tip:** Ensure the file path is correct; handle potential file permissions errors.
 - **Sample Output:** (No output; check the file to see the written text.)
-

16. File Handling: Reading from a File

python

```
with open("sample.txt", "r") as file:  
    content = file.read()  
    print("File content:", content)
```

Explanation:

- **Logic:** Opens a file in read mode and prints its content.
- **Keywords:** `with`, `open`, `read`, `print`
- **Debugging Tip:** Ensure the file exists; handle exceptions for missing files.

Sample Output:

File content: Hello, World!

-
-

17. Linear Search Algorithm

python

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

numbers = [10, 20, 30, 40, 50]
target = 30
result = linear_search(numbers, target)
print("Target found at index:", result)
```

Explanation:

- **Logic:** Implements a linear search algorithm to find the index of a target value in a list.
- **Keywords:** `def`, `for`, `if`, `return`, `print`
- **Debugging Tip:** Check the condition inside the loop for correct matching.

Sample Output:

perl

Target found at index: 2

-
-

18. Bubble Sort Algorithm

python

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
```

```
        arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

numbers = [64, 34, 25, 12, 22]
sorted_numbers = bubble_sort(numbers)
print("Sorted array:", sorted_numbers)
```

Explanation:

- **Logic:** Implements the bubble sort algorithm to sort an array in ascending order.
- **Keywords:** `def`, `for`, `if`, `return`, `print`
- **Debugging Tip:** Verify the swapping logic to ensure proper sorting.

Sample Output:

c

```
Sorted array: [12, 22, 25, 34, 64]
```

-

19. Binary Search Algorithm

python

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] < target:
            low = mid + 1
        elif arr[mid] > target:
            high = mid - 1
        else:
            return mid
    return -1

sorted_numbers = [10, 20, 30, 40, 50]
target = 40
```

```
result = binary_search(sorted_numbers, target)
print("Target found at index:", result)
```

Explanation:

- **Logic:** Implements a binary search algorithm to find the index of a target value in a sorted list.
- **Keywords:** `def`, `while`, `if`, `elif`, `return`, `print`
- **Debugging Tip:** Ensure the list is sorted before applying binary search.

Sample Output:

perl

Target found at index: 3

•

20. Using Functions for Recursion

python

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

number = 5
result = factorial(number)
print("Factorial of", number, "is:", result)
```

Explanation:

- **Logic:** Recursively calculates the factorial of a given number.
- **Keywords:** `def`, `if`, `else`, `return`, `print`
- **Debugging Tip:** Verify base cases to prevent infinite recursion.

Sample Output:

Factorial of 5 is: 120

-

21. Using Lambda Functions

python

```
square = lambda x: x ** 2
print("Square of 5 is:", square(5))
```

Explanation:

- **Logic:** Defines a simple lambda function to calculate the square of a number.
- **Keywords:** `lambda`, `print`
- **Debugging Tip:** Ensure the lambda expression is correctly formed to avoid syntax errors.

Sample Output:

Square of 5 is: 25

-

22. Using Modules

python

```
import math

number = 16
sqrt_value = math.sqrt(number)
print("Square root of", number, "is:", sqrt_value)
```

Explanation:

- **Logic:** Uses the `math` module to calculate the square root of a number.

- **Keywords:** `import`, `print`
- **Debugging Tip:** Ensure the module is correctly imported; check for typos in function names.

Sample Output:

Square root of 16 is: 4.0

-

23. Exception Handling Example

python

```
try:
    num = int(input("Enter a number: "))
    print("You entered:", num)
except ValueError:
    print("That's not a valid number!")
```

Explanation:

- **Logic:** Attempts to convert user input to an integer, handling any `ValueError` exceptions.
- **Keywords:** `try`, `except`, `print`
- **Debugging Tip:** Check the exception handling to ensure proper responses to errors.

Sample Input:

CSS

Enter a number: abc

-

Sample Output:

That's not a valid number!

-

24. Using Python Sets for Unique Items

python

```
my_set = {1, 2, 3, 4, 5, 1, 2}
print("Unique items in the set:", my_set)
```

Explanation:

- **Logic:** Demonstrates how sets automatically handle duplicates.
- **Keywords:** `print`
- **Debugging Tip:** Check the set's properties to ensure it contains unique items.

Sample Output:

sql

```
Unique items in the set: {1, 2, 3, 4, 5}
```

-

25. Tuples and Their Immutability

python

```
my_tuple = (1, 2, 3)
print("Tuple:", my_tuple)
print("First element:", my_tuple[0])
```

Explanation:

- **Logic:** Initializes a tuple and accesses its elements.
- **Keywords:** `print`
- **Debugging Tip:** Remember that tuples are immutable; attempts to modify will result in errors.

Sample Output:

yaml

```
Tuple: (1, 2, 3)
First element: 1
```

-

26. Reading Lines from a File

python

```
with open("sample.txt", "r") as file:
    lines = file.readlines()
    for line in lines:
        print(line.strip())
```

Explanation:

- **Logic:** Reads all lines from a file and prints them one by one.
- **Keywords:** `with`, `open`, `readlines`, `print`
- **Debugging Tip:** Check file existence and permissions if an error occurs.

Sample Output: (Assuming `sample.txt` contains "Hello, World!")

```
Hello, World!
```

-

27. String Manipulation

python

```
text = "Hello, World!"
print("Uppercase:", text.upper())
print("Lowercase:", text.lower())
print("Split:", text.split(","))
```

Explanation:

- **Logic:** Demonstrates various string methods for manipulation.
- **Keywords:** `print`
- **Debugging Tip:** Ensure the string methods are appropriate for the intended manipulation.

Sample Output:

```
Uppercase: HELLO, WORLD!  
Lowercase: hello, world!  
Split: ['Hello', ' World!']
```

-
-

28. Working with Lists: Append and Remove

python

```
fruits = ["apple", "banana"]  
fruits.append("cherry")  
print("After append:", fruits)  
fruits.remove("banana")  
print("After remove:", fruits)
```

Explanation:

- **Logic:** Demonstrates how to append and remove items from a list.
- **Keywords:** `append`, `remove`, `print`
- **Debugging Tip:** Check for the existence of an item before removing to avoid errors.

Sample Output:

less

```
After append: ['apple', 'banana', 'cherry']  
After remove: ['apple', 'cherry']
```

-
-

29. Count Vowels in a String

```
def count_vowels(s):  
    vowels = "aeiouAEIOU"  
    count = 0
```

```
def count_vowels(s):
    for char in s:
        if char in vowels:
            count += 1
    return count

input_string = "Hello World"
print("Number of vowels:", count_vowels(input_string))
```

Explanation:

- **Logic:** Counts the number of vowels in a given string.
- **Keywords:** `def`, `for`, `if`, `return`, `print`
- **Debugging Tip:** Ensure the vowel check is case-insensitive for comprehensive counting.

Sample Output:

javascript

Number of vowels: 3

-

30. Palindrome Check

python

```
def is_palindrome(s):
    return s == s[::-1]

input_string = "madam"
print("Is the string a palindrome?", is_palindrome(input_string))
```

Explanation:

- **Logic:** Checks if a string reads the same forwards and backwards.
- **Keywords:** `def`, `return`, `print`
- **Debugging Tip:** Ensure the input string is stripped of spaces for accurate checks.

Sample Output:

Is the string a palindrome? True

-

31. Fibonacci Sequence

python

```
def fibonacci(n):
    fib_sequence = []
    a, b = 0, 1
    for _ in range(n):
        fib_sequence.append(a)
        a, b = b, a + b
    return fib_sequence

n_terms = 10
print("Fibonacci sequence:", fibonacci(n_terms))
```

Explanation:

- **Logic:** Generates the Fibonacci sequence up to `n` terms.
- **Keywords:** `def`, `for`, `append`, `print`
- **Debugging Tip:** Check the range to avoid generating more terms than needed.

Sample Output:

less

```
Fibonacci sequence: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

-

32. Using JSON to Serialize Data

python

```
import json
```

```
data = {"name": "Alice", "age": 30}
json_data = json.dumps(data)
print("JSON data:", json_data)
```

Explanation:

- **Logic:** Serializes a Python dictionary into a JSON string using the `json` module.
- **Keywords:** `import`, `json.dumps`, `print`
- **Debugging Tip:** Ensure that the data structure is JSON serializable.

Sample Output:

CSS

```
JSON data: {"name": "Alice", "age": 30}
```

-

33. Basic Web Scraping (Using Requests)

python

```
import requests

response = requests.get("https://api.github.com")
print("Status code:", response.status_code)
```

Explanation:

- **Logic:** Fetches data from a URL using the `requests` module.
- **Keywords:** `import`, `requests.get`, `print`
- **Debugging Tip:** Check the status code for successful responses (200 OK).

Sample Output:

CSS

```
Status code: 200
```

-

34. Using Regular Expressions

python

```
import re

pattern = r"\d+"
text = "There are 2 apples and 10 oranges."
numbers = re.findall(pattern, text)
print("Numbers found:", numbers)
```

Explanation:

- **Logic:** Uses regular expressions to find all numeric values in a string.
- **Keywords:** `import`, `re.findall`, `print`
- **Debugging Tip:** Ensure the regular expression pattern is correctly defined for the intended match.

Sample Output:

less

```
Numbers found: ['2', '10']
```

-
-

35. Simple GUI with Tkinter

python

```
import tkinter as tk

def on_click():
    label.config(text="Button clicked!")

root = tk.Tk()
label = tk.Label(root, text="Hello, Tkinter!")
label.pack()
button = tk.Button(root, text="Click Me", command=on_click)
button.pack()
root.mainloop()
```

Explanation:

- **Logic:** Creates a simple GUI application with a label and button using Tkinter.
 - **Keywords:** `import`, `def`, `tk.Tk()`, `tk.Label`, `tk.Button`, `tk.mainloop()`
 - **Debugging Tip:** Ensure Tkinter is properly installed; check for any window display issues.
 - **Sample Output:** (A GUI window appears with a label and button.)
-

36. Object-Oriented Programming: Class and Object

python

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        return f"{self.name} says woof!"

dog = Dog("Buddy")
print(dog.bark())
```

Explanation:

- **Logic:** Defines a class `Dog` with a constructor and method, creating an object.
- **Keywords:** `class`, `def`, `self`, `return`
- **Debugging Tip:** Ensure methods are correctly defined to avoid attribute errors.

Sample Output:

Buddy says woof!

-
-

37. Inheritance Example

python

```
class Animal:
    def speak(self):
        return "Animal speaks"

class Dog(Animal):
    def bark(self):
        return "Dog barks"

dog = Dog()
print(dog.speak())
print(dog.bark())
```

Explanation:

- **Logic:** Demonstrates inheritance with a base class `Animal` and a derived class `Dog`.
- **Keywords:** `class`, `def`, `return`
- **Debugging Tip:** Ensure the parent class is properly referenced to avoid issues.

Sample Output:

```
Animal speaks
Dog barks
```

-
-

38. Polymorphism with Method Overriding

python

```
class Cat:
    def speak(self):
        return "Meow"

class Dog:
    def speak(self):
        return "Woof"

def animal_sound(animal):
    print(animal.speak())
```

```
cat = Cat()
dog = Dog()
animal_sound(cat)
animal_sound(dog)
```

Explanation:

- **Logic:** Demonstrates polymorphism by calling the same method on different objects.
- **Keywords:** `class`, `def`, `print`
- **Debugging Tip:** Ensure the method names are consistent for correct polymorphic behavior.

Sample Output:

```
Meow
Woof
```

•

39. Creating a Simple API with Flask

python

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/api', methods=['GET'])
def api():
    return jsonify({"message": "Hello, World!"})

if __name__ == '__main__':
    app.run(debug=True)
```

Explanation:

- **Logic:** Sets up a simple web API using Flask to respond with JSON data.
- **Keywords:** `from`, `import`, `@app.route`, `def`, `if __name__`, `app.run()`

- **Debugging Tip:** Ensure Flask is installed; check for correct route definition.
 - **Sample Output:** (Accessing <http://127.0.0.1:5000/api> returns {"message": "Hello, World!"})
-

40. Multi-threading Example

python

```
import threading
import time

def print_numbers():
    for i in range(5):
        print(i)
        time.sleep(1)

thread = threading.Thread(target=print_numbers)
thread.start()
thread.join()
```

Explanation:

- **Logic:** Demonstrates multi-threading by running a function in a separate thread.
- **Keywords:** `import`, `threading.Thread`, `start`, `join`
- **Debugging Tip:** Ensure thread safety when accessing shared resources.

Sample Output:

```
0
1
2
3
4
```

-